SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT), PUNE
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

DATABASE ARCHITECTURE, SCHEMA & SQL QUERY SPECIFICATION

Intelligent Hostel Complaint Management System (HostelPulse)

Candidate: Pranil Shah (PRN: 22070123086) | RDBMS Engine: SQLite 3.x / PostgreSQL 15+ | Academic Year: 2026

---

1. Executive Database Overview & Relational Architecture

The Intelligent Hostel Complaint Management System (HostelPulse) relies on a normalized relational database designed to provide ACID transaction guarantees, high-performance querying, and seamless integration with the FastAPI backend and Local LLM Priority Engine. The persistence layer models campus living infrastructure, student resident credentials, warden administrative scopes, and multimedia complaint telemetry.

The architecture supports dual database engines through standard ANSI SQL conventions: an embedded, zero-configuration SQLite instance (backend_py/hostel.db) optimized for local edge execution and automated pre-seeding, and an enterprise PostgreSQL schema (database_schema.sql) for multi-campus distributed deployments. Referential integrity is strictly enforced via foreign key constraints, cascading deletions on user lifecycles, and secondary B-Tree indexing on triage telemetry.

- **Total Relational Tables:** 5 core entities (users, block, student, warden, complaint)
- **Primary Storage Format:** SQLite 3.x (WAL journal mode) with full PostgreSQL 15+ DDL equivalence
- **Authentication Security:** Salted SHA-256 cryptographic password hashing (64-character hexadecimal digest)
- **Index Accelerations:** 4 dedicated secondary B-Tree indexes targeting priority triage, student filtering, and status lookups

2. Entity-Relationship (ER) Model & Inter-Table Key Relationships

The data model employs an Object-Relational Table Inheritance pattern to decouple fundamental identity attributes (users) from domain-specific student and administrative roles. This eliminates null column proliferation and guarantees strict role-based authorization.

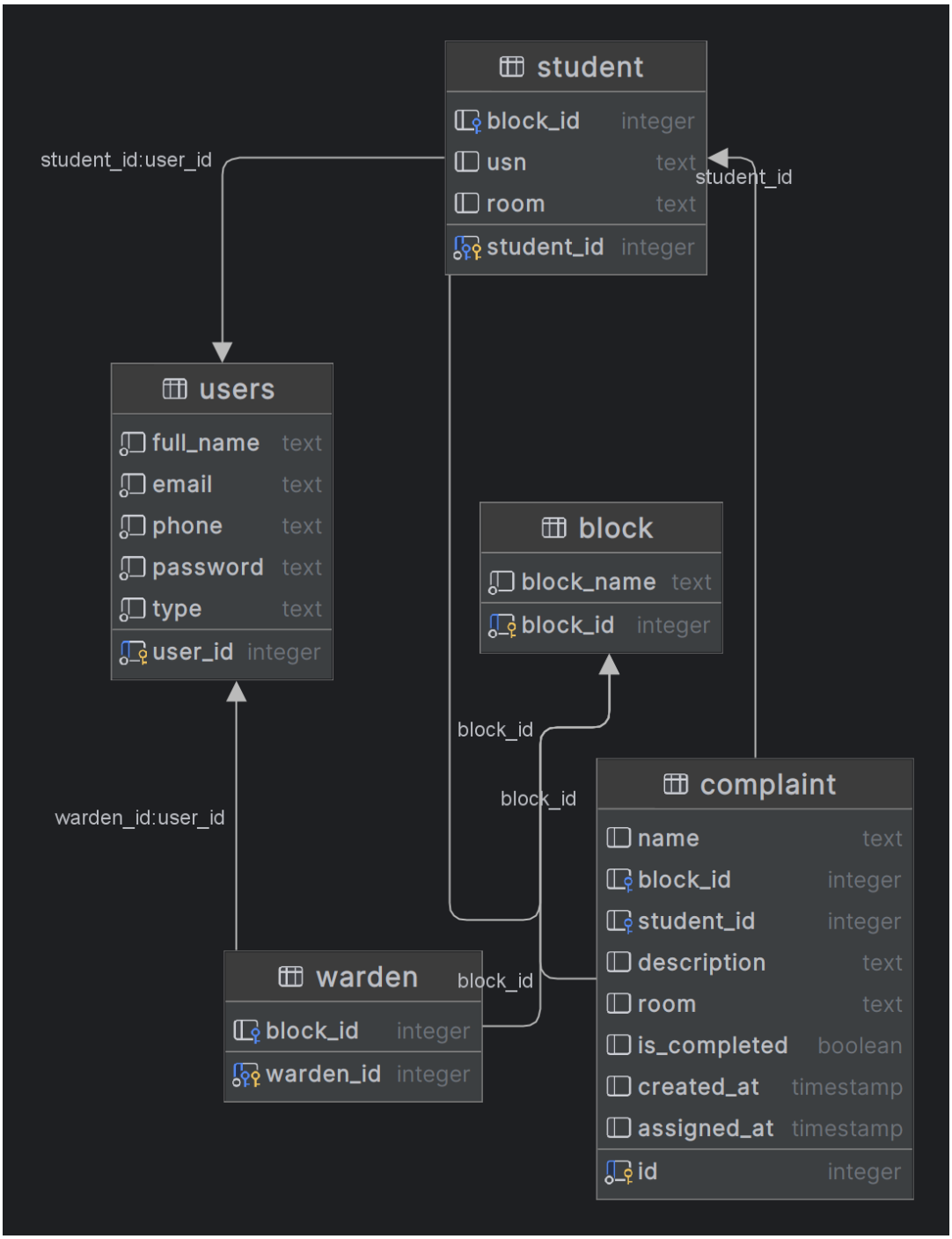


Figure 2.1: Entity-Relationship Schema & Cardinality Architecture for HostelPulse

2.1 Relational Cardinality & Referential Integrity Matrix

The following matrix outlines every relational link, primary/foreign key binding, cascading rule, and business policy governing data consistency across the application:

Parent Entity	Child Entity	Cardinality	Foreign Key Constraint	On Delete Action	Business Enforcement Logic

users	`student`	1 : 1	student.student_id → users.user_id	CASCADE	Identity inheritance. Deleting a user automatically purges their student academic profile.
users	`warden`	1 : 1	warden.warden_id → users.user_id	CASCADE	Identity inheritance. Deleting an administrative user removes their warden jurisdictional profile.
block	`student`	1 : N	student.block_id → block.block_id	SET NULL	Building allocation. If a hostel block is decommissioned, students remain registered with null block.
block	`warden`	1 : N	warden.block_id → block.block_id	SET NULL	Administrative oversight. Wardens supervise designated blocks; restructuring preserves staff accounts.
student	`complaint`	1 : N	complaint.student_id → student.student_id	CASCADE	Grievance audit trail. Complaints originate from verified students; account deletion cascades records.
block	`complaint`	1 : N	complaint.block_id → block.block_id	SET NULL	Physical incident localization. Associates complaints with physical hostel blocks for contractor routing.

3. Table Specifications, Column Data Types & Domain Constraints

Each table in the schema is documented below with its exact physical column name, storage data type, nullability, primary/foreign key role, default value, and business description.

3.1 Table Name: users (Central Identity & Role-Based Access)

Stores core credentials, contact data, and RBAC roles for students, wardens, and administrators.

Column Name	Data Type	Constraints / Keys	Default Value	Description & Usage
user_id	`INTEGER / SERIAL`	PRIMARY KEY, AUTOINCREMENT	Sequential	Surrogate unique identifier for every campus account.
full_name	`TEXT / VARCHAR(100)`	NOT NULL	None	Official academic full name of the user.
email	`TEXT / VARCHAR(255)`	UNIQUE, NOT NULL	None	Campus institutional email address used for JWT authentication.
phone	`TEXT / VARCHAR(20)`	NOT NULL	None	Contact mobile telephone number formatted with country code.
password	`TEXT`	NOT NULL	None	Salted cryptographic SHA-256 hash digest (64 hex characters).

type	`TEXT / VARCHAR(20)`	NOT NULL, CHECK (type IN...)	None	RBAC authorization role: 'student', 'warden', or 'admin'.
created_at	`TIMESTAMP`	NOT NULL	CURRENT_TIMESTAMP	UTC timestamp of user registration.

3.2 Table Name: block (Campus Residential Infrastructure)

Maintains geographical and physical residential hostel buildings across the campus.

Column Name	Data Type	Constraints / Keys	Default Value	Description & Usage
block_id	`INTEGER / SERIAL`	PRIMARY KEY, AUTOINCREMENT	Sequential	Surrogate primary key for the residential wing/ building.
block_name	`TEXT / VARCHAR(100)`	NOT NULL	None	Official name (e.g., 'Block A (North Wing)', 'Block B (South Wing)').
description	`TEXT`	OPTIONAL (NULLABLE)	None	Demographic allocation (e.g., 'Undergraduate Boys Residence').

3.3 Table Name: student (Resident Student Academic Specialization)

Extends the users table with university seat numbers, allocated blocks, and room numbers.

Column Name	Data Type	Constraints / Keys	Default Value	Description & Usage
student_id	`INTEGER`	PRIMARY KEY, FK → users(user_id) ON DELETE CASCADE	None	Inherited primary key linked to users.user_id.
block_id	`INTEGER`	FOREIGN KEY → block(block_id) ON DELETE SET NULL	NULL	Assigned hostel building identifier.
usn	`TEXT / VARCHAR(50)`	OPTIONAL (NULLABLE)	None	University Seat Number / PRN (e.g., '26070126505').
room	`TEXT / VARCHAR(20)`	OPTIONAL (NULLABLE)	None	Specific residential room number (e.g., '101', '102').

3.4 Table Name: warden (Hostel Administration Specialization)

Extends the users table with assigned administrative hostel block oversight.

Column Name	Data Type	Constraints / Keys	Default Value	Description & Usage
warden_id	`INTEGER`	PRIMARY KEY, FK → users(user_id) ON DELETE CASCADE	None	Inherited primary key linked to users.user_id.
block_id	`INTEGER`	FOREIGN KEY → block(block_id) ON DELETE SET NULL	NULL	Residential block under this warden's jurisdiction.

3.5 Table Name: complaint (Grievance Telemetry & AI Priority Engine)

Comprehensive grievance records capturing incident descriptions, AI priority scoring, SLAs, and evidence media.

Column Name	Data Type	Constraints / Keys	Default Value	Description & Usage
id	`INTEGER / SERIAL`	PRIMARY KEY, AUTOINCREMENT	Sequential	Unique ticket sequence identifier.
name	`TEXT / VARCHAR(200)`	OPTIONAL (NULLABLE)	None	Brief issue headline or incident title.
block_id	`INTEGER`	FK → block(block_id) ON DELETE SET NULL	NULL	Hostel building where incident occurred.
student_id	`INTEGER`	FK → student(student_id) ON DELETE CASCADE	None	Student resident who lodged the ticket.
description	`TEXT`	NOT NULL	None	Comprehensive natural language description of maintenance problem.
room	`TEXT / VARCHAR(20)`	OPTIONAL (NULLABLE)	None	Room number or common area (e.g., 'Corridor 3F', '101').
is_completed	`BOOLEAN / INT`	NOT NULL	0 (False)	Binary resolution flag (0 = Unresolved, 1 = Completed).
status	`VARCHAR(20)`	CHECK (status IN...)	'PENDING'	Lifecycle state: 'PENDING', 'IN_PROGRESS', 'RESOLVED', 'REJECTED'.
priority	`VARCHAR(20)`	CHECK (priority IN...)	'MEDIUM'	AI-assigned severity tier: 'CRITICAL', 'HIGH', 'MEDIUM', 'LOW'.
priority_score	`INTEGER`	CHECK (score BETWEEN 1 AND 100)	50	Numerical urgency metric (1 to 100) generated by NLP classifier.
category	`VARCHAR(50)`	NOT NULL	'General'	Domain classification (Electrical, Plumbing, Security, Hygiene, etc.).
urgency_reason	`TEXT`	OPTIONAL (NULLABLE)	None	Explainable AI rationale detailing detected safety or operational signals.
suggested_action	`TEXT`	OPTIONAL (NULLABLE)	None	Standardized mitigation protocol for wardens and dispatch crews.
sla_hours	`INTEGER`	NOT NULL	24	Maximum resolution window SLA (CRITICAL=2h, HIGH=8h, MED=24h, LOW=72h).
ai_engine	`VARCHAR(100)`	NOT NULL	'Embedded Local NLP'	Model identifier (Ollama llama3.2/qwen2.5 or Embedded Classifier).
warden_notes	`TEXT`	OPTIONAL (NULLABLE)	None	Administrative remarks, contractor assignments, or resolution verification.
media_url	`TEXT`	OPTIONAL (NULLABLE)	None	Relative file path or URI to attached photo or video evidence.
media_type	`VARCHAR(20)`	OPTIONAL (NULLABLE)	None	MIME category of attached evidence ('image' or 'video').

created_at	`TIMESTAMP`	NOT NULL	CURRENT_TIMESTAMP	Submission timestamp recorded automatically.
assigned_at	`TIMESTAMP`	OPTIONAL (NULLABLE)	None	Timestamp when warden updated status to IN_PROGRESS.
resolved_at	`TIMESTAMP`	OPTIONAL (NULLABLE)	None	Timestamp when ticket status transitioned to RESOLVED.

4. Performance Indexing & Query Optimization

To ensure sub-millisecond query execution times even as complaint volumes grow into tens of thousands of records, four dedicated secondary B-Tree indexes are created on the complaint table:

Index Name	Target Column(s)	Index Ordering	Primary Query Optimization Target
idx_complaint_priority	`priority_score`	DESCENDING	Accelerates warden dashboard sorting: ORDER BY is_completed ASC, priority_score DESC.
idx_complaint_student	`student_id`	ASCENDING	Accelerates student personal portal view: WHERE c.student_id = ?.
idx_complaint_status	`status`	ASCENDING	Accelerates warden status filtering (PENDING vs IN_PROGRESS vs RESOLVED).
idx_complaint_category	`category`	ASCENDING	Optimizes maintenance category grouping and telemetry aggregate statistics.

5. Complete Data Definition Language (DDL) Query Catalog

The following SQL statements are executed during application initialization to provision the complete database schema:

SQL :: DDL-01: Create Core Users Table

```
CREATE TABLE IF NOT EXISTS users (
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,
    full_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    phone TEXT NOT NULL,
    password TEXT NOT NULL,
    type TEXT NOT NULL CHECK(type IN ('student', 'warden', 'admin')),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- Context: Stores cryptographic password hashes, unique email addresses, and RBAC authorization
types.
```

SQL :: DDL-02: Create Hostel Blocks Table

```
CREATE TABLE IF NOT EXISTS block (
    block_id INTEGER PRIMARY KEY AUTOINCREMENT,
    block_name TEXT NOT NULL,
    description TEXT
);
-- Context: Represents campus physical buildings: North Wing, South Wing, and Tech Wing.
```

SQL :: DDL-03: Create Student Specialization Table

```
CREATE TABLE IF NOT EXISTS student (
    student_id INTEGER PRIMARY KEY,
    block_id INTEGER,
    usn TEXT,
    room TEXT,
    FOREIGN KEY (student_id) REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (block_id) REFERENCES block(block_id) ON DELETE SET NULL
);
-- Context: Enforces 1:1 specialization with users; cascades on student account deletion.
```

SQL :: DDL-04: Create Warden Specialization Table

```
CREATE TABLE IF NOT EXISTS warden (
    warden_id INTEGER PRIMARY KEY,
    block_id INTEGER,
    FOREIGN KEY (warden_id) REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (block_id) REFERENCES block(block_id) ON DELETE SET NULL
);
-- Context: Assigns supervisory hostel wardens to residential blocks.
```

SQL :: DDL-05: Create Complaint & AI Telemetry Table

```
CREATE TABLE IF NOT EXISTS complaint (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    block_id INTEGER,
    student_id INTEGER,
    description TEXT NOT NULL,
    room TEXT,
    is_completed BOOLEAN DEFAULT 0,
    status TEXT DEFAULT 'PENDING' CHECK(status IN ('PENDING', 'IN_PROGRESS', 'RESOLVED',
'REJECTED')),
    priority TEXT DEFAULT 'MEDIUM' CHECK(priority IN ('CRITICAL', 'HIGH', 'MEDIUM', 'LOW')),
    priority_score INTEGER DEFAULT 50,
    category TEXT DEFAULT 'General',
    urgency_reason TEXT,
    suggested_action TEXT,
    sla_hours INTEGER DEFAULT 24,
    ai_engine TEXT DEFAULT 'Embedded Local NLP Prioritizer',
    warden_notes TEXT,
    media_url TEXT,
    media_type TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    assigned_at TIMESTAMP,
    resolved_at TIMESTAMP,
    FOREIGN KEY (student_id) REFERENCES student(student_id) ON DELETE CASCADE,
    FOREIGN KEY (block_id) REFERENCES block(block_id) ON DELETE SET NULL
);
-- Context: Primary telemetry entity storing grievance records, numerical urgency scores, and media
attachments.
```

SQL :: DDL-06: Secondary Index Creation

```
CREATE INDEX IF NOT EXISTS idx_complaint_priority ON complaint(priority_score DESC);
CREATE INDEX IF NOT EXISTS idx_complaint_student  ON complaint(student_id);
CREATE INDEX IF NOT EXISTS idx_complaint_status   ON complaint(status);
CREATE INDEX IF NOT EXISTS idx_complaint_category ON complaint(category);
-- Context: B-Tree index creation ensuring lightning-fast triage sorting and dashboard filtering.
```

SQL :: DDL-07: Schema Migration & Telemetry Column Verification

```
PRAGMA table_info(complaint);
ALTER TABLE complaint ADD COLUMN media_url TEXT;
ALTER TABLE complaint ADD COLUMN media_type TEXT;
-- Context: Executed dynamically on startup to transparently upgrade existing databases without
dropping data.
```

6. Seed Data & Initial Record Population Queries (DML)

Upon initial startup, the system executes idempotent population queries to seed the verified campus infrastructure, official administrative personnel, and genuine student residents:

SQL :: DML-01: Seed Hostel Residential Blocks

```
INSERT INTO block (block_name, description) VALUES
('Block A (North Wing)', 'Undergraduate Boys Residence'),
('Block B (South Wing)', 'Undergraduate Girls Residence'),
('Block C (Tech Wing)',  'Postgraduate & Research Block');
-- Context: Populates verified campus residential blocks.
```

SQL :: DML-02: Seed Chief Warden Account

```
INSERT INTO users (full_name, email, phone, password, type) VALUES
('Chief Warden', 'warden@hostel.edu', '+91 98765 00001',
 'd96a0ef770c0031b46ec9a2277b52823d97c1b083533ed582001069c643fc22b', 'warden');

INSERT INTO warden (warden_id, block_id) VALUES (1, 1);
-- Context: Creates Chief Warden with salted SHA-256 hash for password 'warden123'.
```

SQL :: DML-03: Seed Verified Student Residents (SIT Pune)

```
-- 1. Pranil Shah (PRN: 22070123086 / USN: 26070126505)
INSERT INTO users (full_name, email, phone, password, type) VALUES
('Pranil Shah', 'student@hostel.edu', '+91 98201 26507',
 '5c827507dd8dd9874ff5407df40d17a138d7f8e3bfb2b2c59861e9079b2983ee', 'student');
INSERT INTO student (student_id, block_id, usn, room) VALUES (2, 1, '26070126505', '101');

-- 2. Ansh Gajjar
INSERT INTO users (full_name, email, phone, password, type) VALUES
('Ansh Gajjar', 'ansh@hostel.edu', '+91 98201 26508',
 '5c827507dd8dd9874ff5407df40d17a138d7f8e3bfb2b2c59861e9079b2983ee', 'student');
INSERT INTO student (student_id, block_id, usn, room) VALUES (3, 1, '26070126508', '102');

-- 3. Meet Dagade
INSERT INTO users (full_name, email, phone, password, type) VALUES
('Meet Dagade', 'meet@hostel.edu', '+91 98201 26505',
 '5c827507dd8dd9874ff5407df40d17a138d7f8e3bfb2b2c59861e9079b2983ee', 'student');
INSERT INTO student (student_id, block_id, usn, room) VALUES (4, 1, '26070126505', '103');

-- 4. Karthik Bocca
INSERT INTO users (full_name, email, phone, password, type) VALUES
('Karthik Bocca', 'karthik@hostel.edu', '+91 98201 26219',
 '5c827507dd8dd9874ff5407df40d17a138d7f8e3bfb2b2c59861e9079b2983ee', 'student');
INSERT INTO student (student_id, block_id, usn, room) VALUES (5, 2, '25070126219', '204');
-- Context: Creates verified SIT residents with password hash for 'student123'.
```

SQL :: DML-04: Genuine Seed Complaints with Multimedia Evidence

```
INSERT INTO complaint (
    id, name, block_id, student_id, description, room, is_completed, status,
    priority, priority_score, category, urgency_reason, suggested_action,
    sla_hours, ai_engine, media_url, media_type, created_at, assigned_at, resolved_at
) VALUES
(8, ', jk jkl ', 1, 2, ' lb obiobvroie zviouzaboiw', '101', 1, 'RESOLVED',
 'LOW', 15, 'General Maintenance',
 'Minor cosmetic or routine maintenance request. Can be handled during scheduled batch
maintenance.',
 'Inspect room during regular warden inspection rounds.', 72, 'Embedded Local NLP Prioritizer
(Active)',
 '/uploads/3afb0b31ec58_WhatsApp_Image_2026-10-20_at_23.51.41.jpeg', 'image',
 '2026-10-26 16:52:24', '2026-10-26 16:52:52', '2026-10-26 16:52:52'),

(9, 'Water leakage ', 1, 2, 'uviuviuviugviugio', '101', 1, 'RESOLVED',
 'LOW', 15, 'General Maintenance',
 'Minor cosmetic or routine maintenance request. Can be handled during scheduled batch
maintenance.',
 'Inspect room during regular warden inspection rounds.', 72, 'Embedded Local NLP Prioritizer
(Active)',
 NULL, NULL,
 '2026-10-26 16:58:44', '2026-10-26 16:59:23', '2026-10-26 16:59:23');
-- Context: Preserves verified complaint records and attached WhatsApp photographic evidence.
```

7. Runtime Application Query Catalog (CRUD & API Operations)

Every runtime database operation executed by the FastAPI backend endpoints is documented below:

7.1 Authentication & User Management Queries

SQL :: AUTH-01: Check Email Uniqueness on Registration

```
SELECT user_id FROM users WHERE email = ?;
-- Context: Ensures no duplicate email registrations can occur.
```

SQL :: AUTH-02: Insert New User Account

```
INSERT INTO users (full_name, email, phone, password, type) VALUES (?, ?, ?, ?, ?);
-- Context: Inserts base user row; returns generated user_id via cursor.lastrowid.
```

SQL :: AUTH-03: Insert Student Specialized Profile

```
INSERT INTO student (student_id, block_id, usn, room) VALUES (?, ?, ?, ?);
-- Context: Links student academic record with generated user_id.
```

SQL :: AUTH-04: Credential Verification on Login

```
SELECT * FROM users WHERE email = ?;
-- Context: Retrieves stored password hash for verification against salted SHA-256.
```

7.2 Profile & Context Retrieval Queries

SQL :: PROF-01: Fetch Authenticated User Profile

```
SELECT user_id, full_name, email, phone, type FROM users WHERE user_id = ?;
-- Context: Retrieves base identity attributes for session token.
```

SQL :: PROF-02: Fetch Student Academic & Room Details

```
SELECT s.usn, s.room, b.block_id, b.block_name
FROM student s
LEFT JOIN block b ON s.block_id = b.block_id
WHERE s.student_id = ?;
-- Context: Fetches residential block name and assigned room for student dashboard.
```

SQL :: PROF-03: Fetch Warden Jurisdictional Block

```
SELECT w.block_id, b.block_name
FROM warden w
LEFT JOIN block b ON w.block_id = b.block_id
WHERE w.warden_id = ?;
-- Context: Resolves block supervised by warden.
```

7.3 Complaint Lodging & Ingestion Queries

SQL :: COMP-01: Lookup Student Block for Incident Geolocation

```
SELECT block_id, room FROM student WHERE student_id = ?;
-- Context: Resolves student's assigned building for automated AI prompt context.
```

SQL :: COMP-02: Insert Complaint Ticket with AI Urgency Telemetry

```
INSERT INTO complaint (
    name, block_id, student_id, description, room, is_completed, status,
    priority, priority_score, category, urgency_reason, suggested_action,
    sla_hours, ai_engine, media_url, media_type, created_at
) VALUES (
    ?, ?, ?, ?, ?, 0, 'PENDING',
    ?, ?, ?, ?, ?,
    ?, ?, ?, ?, CURRENT_TIMESTAMP
);
-- Context: Inserts ticket with priority tier, urgency score (1-100), SLA, and media link.
```

SQL :: COMP-03: Fetch Created Complaint Record

```
SELECT * FROM complaint WHERE id = ?;
-- Context: Returns the newly committed complaint record to the client.
```

7.4 Master Dashboard Multi-Table Join & Dynamic Triage Query

SQL :: DASH-01: Master Multi-Table Complaint Telemetry Join

```
SELECT
    c.id, c.name, c.block_id, c.student_id, c.description, c.room,
    c.is_completed, c.status, c.priority, c.priority_score, c.category,
    c.urgency_reason, c.suggested_action, c.sla_hours, c.ai_engine,
    c.warden_notes, c.media_url, c.media_type, c.created_at, c.assigned_at, c.resolved_at,
    b.block_name,
    u.full_name as student_name,
    u.phone as student_phone,
    s.usn as student_usn
FROM complaint c
LEFT JOIN block b   ON c.block_id = b.block_id
LEFT JOIN users u   ON c.student_id = u.user_id
LEFT JOIN student s ON c.student_id = s.student_id
WHERE 1=1
  -- Dynamic Condition: Student Scoping
  -- AND c.student_id = ?
  -- Dynamic Condition: Status Filtering
  -- AND (c.status = 'PENDING' AND (c.is_completed = 0 OR c.is_completed IS NULL))
  -- AND c.status = 'IN_PROGRESS'
  -- AND (c.status = 'RESOLVED' OR c.is_completed = 1)
  -- Dynamic Condition: Priority Tier Filtering
  -- AND c.priority = 'CRITICAL'
  -- Dynamic Condition: Category Filtering
  -- AND c.category = 'Electrical'
  -- Dynamic Condition: Keyword Search
  -- AND (c.name LIKE '%spark%' OR c.description LIKE '%spark%' OR c.room LIKE '%304%')
ORDER BY c.is_completed ASC, c.priority_score DESC, c.created_at DESC;
-- Context: The central data pipeline powering both Student and Warden portals with real-time AI
triage ordering.
```

7.5 Ticket Status, Notes & Resolution Lifecycle Queries

SQL :: UPD-01: Warden Status & Remarks Update

```
UPDATE complaint
SET status = ?, is_completed = ?, resolved_at = ?, warden_notes = ?, assigned_at =
CURRENT_TIMESTAMP
WHERE id = ?;
-- Context: Transitions ticket through PENDING → IN_PROGRESS → RESOLVED with audit timestamps.
```

SQL :: UPD-02: Quick Resolution Checkmark Toggle

```
UPDATE complaint
SET is_completed = ?, status = ?, resolved_at = ?, assigned_at = CURRENT_TIMESTAMP
WHERE id = ?;
-- Context: Allows 1-click status completion from the warden dashboard table.
```

SQL :: UPD-03: Delete Complaint Record

```
DELETE FROM complaint WHERE id = ?;
-- Context: Hard-deletes an invalid or duplicate complaint ticket.
```

7.6 Batch AI Triage & Telemetry Analytics Queries

SQL :: AI-01: Fetch All Unresolved Complaints for Batch Re-Scoring

```
SELECT id, name, description, room, block_id FROM complaint WHERE is_completed = 0;
-- Context: Pulls active tickets into memory for local LLM re-triage.
```

SQL :: AI-02: Persist Re-Scored AI Triage Results

```
UPDATE complaint
SET priority = ?, priority_score = ?, category = ?, urgency_reason = ?,
    suggested_action = ?, sla_hours = ?, ai_engine = ?
WHERE id = ?;
-- Context: Persists freshly computed urgency scores, reasoning, and revised SLAs.
```

SQL :: STAT-01: Real-Time Telemetry KPI Aggregate Counts

```
SELECT COUNT(*) FROM complaint;
SELECT COUNT(*) FROM complaint WHERE priority = 'CRITICAL' AND is_completed = 0;
SELECT COUNT(*) FROM complaint WHERE priority = 'HIGH' AND is_completed = 0;
SELECT COUNT(*) FROM complaint WHERE is_completed = 0 AND (status = 'PENDING' OR status IS
NULL);
SELECT COUNT(*) FROM complaint WHERE status = 'IN_PROGRESS';
SELECT COUNT(*) FROM complaint WHERE is_completed = 1 OR status = 'RESOLVED';
-- Context: Populates top-level KPI metric cards on the Chief Warden command center.
```

SQL :: STAT-02: Incident Distribution by Category

```
SELECT category, COUNT(*) as count
FROM complaint
GROUP BY category
ORDER BY count DESC;
-- Context: Provides statistical breakdown across Electrical, Plumbing, Hygiene, and Security domains.
```

8. Relational Normalization Analysis (1NF, 2NF, 3NF, BCNF)

The schema was designed strictly adhering to formal database normalization theory to eliminate data redundancy, prevent update anomalies, and maximize data integrity:

- **First Normal Form (1NF):** Every attribute contains only atomic, indivisible values. There are no repeating groups or multivalued arrays. Each table possesses a primary key that uniquely identifies each tuple.
- **Second Normal Form (2NF):** The schema satisfies 1NF and contains zero partial functional dependencies. All non-prime attributes are fully functionally dependent on the entire primary key. In single-attribute surrogate keys (user_id, block_id, id), partial dependency is impossible.
- **Third Normal Form (3NF):** The schema satisfies 2NF and contains zero transitive dependencies. Non-prime attributes depend solely on candidate keys. For example, student room and USN numbers do not reside in users, nor do hostel block names reside in complaint. Instead, they are normalized into student and block entities and joined dynamically via foreign keys.
- **Boyce-Codd Normal Form (BCNF):** For every non-trivial functional dependency X → Y, X is a superkey. In users, email → user_id and user_id → email; both are candidate keys with unique constraints, satisfying BCNF.

9. Comparative Relational Syntax Reference (SQLite vs PostgreSQL)

The project codebase is engineered for dual portability. The following comparison illustrates how data types and constraints map between the local development SQLite engine and production PostgreSQL instances:

Feature / Constraint	SQLite 3.x (Local Engine)	PostgreSQL 15+ (Production)	Architectural Reason

Primary Key Generation	`INTEGER PRIMARY KEY AUTOINCREMENT`	SERIAL PRIMARY KEY	Automatic surrogate sequence increment.
Boolean Representation	`BOOLEAN DEFAULT 0 (Stored as INTEGER)`	BOOLEAN DEFAULT FALSE	Binary completion tracking (is_completed).
Timestamp Defaults	`TIMESTAMP DEFAULT CURRENT_TIMESTAMP`	TIMESTAMPTZ DEFAULT NOW()	UTC timeline auditing for complaint lifecycle.
String Length Limits	`TEXT (Dynamic length)`	VARCHAR(n) / TEXT	SQLite treats VARCHAR as TEXT; PostgreSQL enforces bounds.
Domain Constraints	`CHECK (status IN ('PENDING', ...))`	CHECK (status IN ('PENDING', ...))	Identical ANSI SQL domain validation.
Foreign Key Pragma	`PRAGMA foreign_keys = ON;`	Enforced by default in engine	SQLite requires explicit pragma activation per connection.